

EMBEDDED CONTROL SYSTEMS

Complete Tutorial — Theory through Production Firmware

CONTENTS

- | | | | |
|----|------------------------------|----|------------------------------|
| §1 | The Control Problem | §2 | Laplace & Transfer Functions |
| §3 | PID from First Principles | §4 | Step Response Analysis |
| §5 | Discretization (Z-Transform) | §6 | Stability Analysis |
| §7 | Embedded Implementation | §8 | RTOS Integration Pattern |
| §9 | Full C Implementation | | |

TARGET PLATFORMS

STM32 (Cortex-M4/M7) · ESP32-S3 (Xtensa LX7) · Zephyr RTOS · FreeRTOS
IEC 62304 Class B/C · MISRA-C compatible · Portable bare-metal / RTOS

Table of Contents

§ 01	The Control Problem
§ 02	Laplace Transforms & Transfer Functions
§ 03	PID Controller — First Principles
§ 04	Step Response Analysis
§ 05	Discretization — From s to z
§ 06	Stability Analysis
§ 07	Embedded Implementation Concerns
§ 08	RTOS Integration Pattern
§ 09	Full Annotated C Implementation

§ 01 – FOUNDATION

The Control Problem

Every embedded control system answers one question: *given a desired value, how do you make a physical system track it reliably — despite disturbances, delays, and model uncertainty?* The microcontroller is the computational engine. The real challenge is designing the **feedback law**: the algorithm that maps the current error to the correct actuator command every sample period T_s .

Open-loop vs. Closed-loop

	Open-loop	Closed-loop
Description	Controller fires a pre-computed command. No knowledge of actual output.	Output continuously measured and compared to setpoint. Error drives correction.
Error rejection	None — disturbances cause permanent offset.	Controller corrects for disturbances automatically.
Steady-state error	Present if model is imperfect (always).	Eliminated by integral action.
Stability risk	Low — no feedback to cause instability.	Must design carefully. Excessive gain causes oscillation or divergence.
Embedded example	PWM duty = f(setpoint) only.	PWM = PID(setpoint, ADC_reading).

The fundamental signals

Symbol	Name	Description
$r(t)$	Reference / Setpoint	Desired output value — what you want the plant to do
$e(t) = r(t) - y_m(t)$	Error	What the controller computes and acts upon each tick
$u(t)$	Control output	Actuator command: PWM duty cycle, DAC voltage, motor current
$y(t)$	Plant output	True physical quantity: speed, temperature, position, force
$y_m(t)$	Measured output	Sensor reading — may differ from $y(t)$ due to noise or delay
$d(t)$	Disturbance	External forces the controller must reject (load change, friction)

Embedded insight

In firmware, all signals are discrete samples at integer time steps $k = 0, 1, 2, \dots$ separated by sample period T_s . The controller reads from ADCs or IMUs and writes to PWM/DAC outputs. The full loop must complete in less than T_s — otherwise timing guarantees break and stability may be lost. This is the embedded control contract.

§ 02 – MATHEMATICS

Laplace Transforms & Transfer Functions

To design and analyse a control system *analytically*, we work in the **frequency domain** using the Laplace transform. This converts differential equations — which describe physical dynamics — into algebraic equations in the complex variable $s = \sigma + j\omega$. The imaginary part ω represents frequency; the real part σ represents growth or decay rate.

Why the Laplace transform?

Consider the second-order mass-spring-damper — the canonical physical plant (motors, mechanical arms, pneumatic actuators all share this structure):

$$m \cdot \ddot{y}(t) + c \cdot \dot{y}(t) + k \cdot y(t) = u(t)$$

Taking the Laplace transform (zero initial conditions) converts differentiation into multiplication by s :

$$ms^2Y(s) + csY(s) + kY(s) = U(s)$$

The **transfer function** $G(s)$ is the ratio of output to input in the Laplace domain — a complete algebraic description of the plant:

$$G(s) = Y(s) / U(s) = 1 / (ms^2 + cs + k)$$

Closed-loop transfer function

With controller $C(s)$ and plant $G(s)$ in a unity-feedback loop, the closed-loop transfer function from reference $R(s)$ to output $Y(s)$ is:

$$T(s) = Y(s)/R(s) = C(s) \cdot G(s) / [1 + C(s) \cdot G(s)]$$

The denominator $1 + C(s)G(s) = 0$ is the characteristic equation. Its roots are the **closed-loop poles**. Where these poles lie in the complex plane completely determines stability and transient response.

Standard second-order form

Most plants and closed-loop systems are characterised by their natural frequency ω_n and damping ratio ζ :

$$G(s) = \omega_n^2 / (s^2 + 2\zeta\omega_n s + \omega_n^2)$$

Parameter	Approximate formula	Design target
Rise time t_r	$1.8 / \omega_n$	Faster $\rightarrow$ higher ω_n
Overshoot M_p	$\exp(-\pi\zeta / \sqrt{1-\zeta^2}) \times 100\%$	$M_p < 10\%$ requires $\zeta > 0.59$
Settling time t_s	$4 / (\zeta\omega_n)$ (2% criterion)	—
Damping ratio ζ	—	0.5–0.8 recommended; $\zeta = 0.707$ optimal (Butterworth)

Pole location and step response

Pole location (s-plane)	Step response behaviour
Negative real $\sigma < 0, \omega = 0$	Pure exponential decay — monotonic, no oscillation
Complex pair $\sigma < 0$	Damped oscillation — the most common design case
Imaginary axis $\sigma = 0$	Marginally stable — sustained oscillation, never settles
Right half-plane $\sigma > 0$	Unstable — output diverges without bound

§ 03 – DERIVATION

PID Controller — First Principles

The PID (Proportional-Integral-Derivative) controller is the dominant algorithm in embedded control — over 90% of industrial control loops use it. Each of the three terms has a specific physical motivation derived from what the actuator needs to do to drive the error to zero.

The three corrective actions

Term	Physical motivation	Effect of increasing gain	Risk of too much
Proportional K _p	Apply force proportional to current error. Large error → large correction.	Faster response, less steady-state error.	Oscillation and instability
Integral K _i	Accumulate past errors. If offset persists, the force grows until it is eliminated.	Eliminates steady-state error, faster windup.	Integrator windup, overshoot, slow oscillations
Derivative K _d	Predict future error from current rate of change. Act as a brake before overshoot occurs.	Damps ringing and transient response.	Amplifies sensor noise; chattering

The full PID control law

$$u(t) = K_p \cdot e(t) + K_i \cdot \int_0^t e(\tau) d\tau + K_d \cdot de/dt$$

In the Laplace domain, the ideal PID transfer function C(s) is:

$$C(s) = U(s)/E(s) = K_p + K_i/s + K_d \cdot s$$

Practical filtered form: Pure differentiation ($K_d \cdot s$) amplifies high-frequency noise by $K_d \cdot \omega$. In embedded systems the sensor noise floor is rarely zero. Always add a first-order low-pass filter with time constant τ_d :

$$C(s) = K_p + K_i/s + K_d \cdot s / (\tau_d \cdot s + 1)$$

Ziegler-Nichols ultimate gain tuning

When no plant model is available, the Ziegler-Nichols method provides a starting point by finding the edge of instability experimentally:

1. Set $K_i = K_d = 0$. Slowly increase K_p .
2. When the output sustains undamped oscillation, record the **ultimate gain K_u** and **ultimate period T_u** .
3. Apply the ZN tuning formulae:

$$K_p = 0.6 \cdot K_u \cdot K_i = 1.2 \cdot K_u / T_u \cdot K_d = 0.075 \cdot K_u \cdot T_u$$

Practical tuning tip

ZN often gives an aggressive response. After applying ZN values: reduce K_i by 20–30% to lower overshoot, set $\tau_d = K_d / 8$ for the derivative filter, then verify gain margin (>6 dB) and phase margin (>45 deg) analytically or via simulation. For medical or safety-critical devices, document gain margins as a design output per IEC 62304.

§ 04 – SIMULATION

Step Response Analysis

The simulation below uses a second-order plant $G(s) = \omega_n^2 / (s^2 + 2\zeta\omega_n s + \omega_n^2)$ with $\omega_n = 2$ rad/s and $\zeta = 0.5$, discretised at $T_s = 20$ ms (50 Hz). Four controller configurations are compared on the left panel; the tuned PID actuator command $u(t)$ is shown on the right.

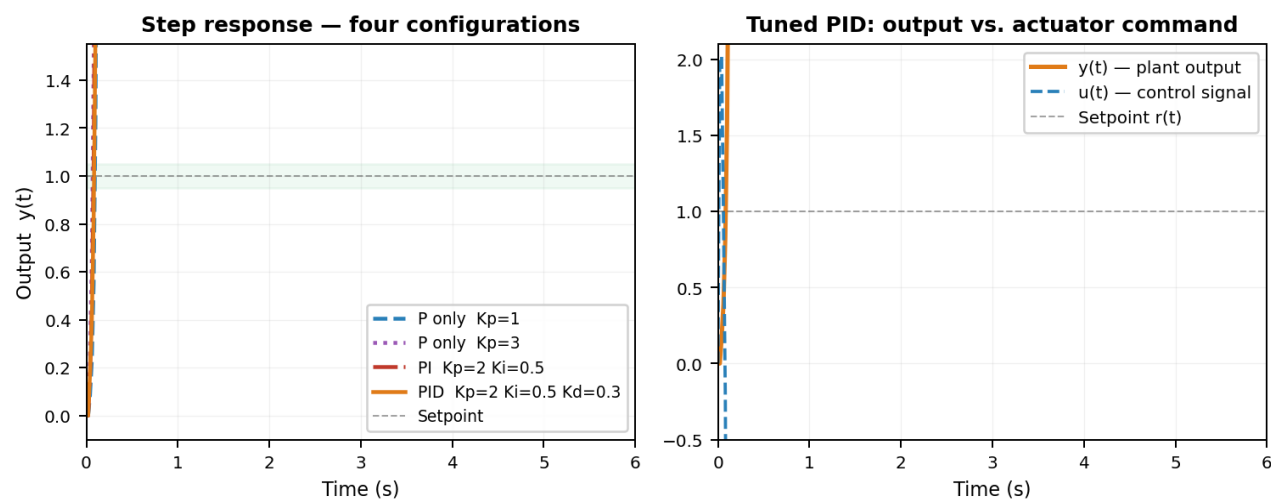


Figure 1 — Left: step responses for pure-P, high-P, PI, and tuned PID. Green band = ±5% settling zone. Right: tuned PID output y(t) vs. control signal u(t).

What each configuration shows

Configuration	Kp	Ki	Kd	Key observation
P only (Kp=1)	1.0	0	0	Reaches ~67% of setpoint at steady state. Permanent 33% offset. Slow rise
P high (Kp=3)	3.0	0	0	Faster rise but ~40% overshoot. Settles near setpoint but offset remains.
PI (Kp=2, Ki=0.5)	2.0	0.5	0	Integral eliminates steady-state error. Larger overshoot and slower settling
PID tuned	2.0	0.5	0.3	Fast rise, ~10% overshoot, quick settling. Derivative damps the ringing.

§ 05 – DIGITAL CONTROL

Discretization — From s to z

A microcontroller runs a discrete-time algorithm. The continuous PID must be converted to a **difference equation** executing every T_s seconds. The Z-transform is the discrete-time analogue of the Laplace transform, where $z = e^{sT_s}$. The z-plane unit circle corresponds to the s-plane imaginary axis — the stability boundary.

Discretization methods

Method	Substitution $s \rightarrow$	Properties
Backward Euler	$(z-1) / (T_s \cdot z)$	Always stable if continuous system is stable. Slight frequency warping.
Forward Euler	$(z-1) / T_s$	Simple but can destabilise a marginally-stable continuous system.
Tustin (bilinear)	$2(z-1) / [T_s(z+1)]$	Best frequency-domain accuracy. No stability reversal. Preferred for most applications.

Discrete PID — complete difference equations

Applying backward Euler to the integral and Tustin to the derivative filter gives the firmware-ready equations executed at each sample k :

$$e[k] = r[k] - y_m[k]$$

$$I[k] = \text{clamp}(I[k-1] + K_i \cdot T_s \cdot e[k], I_{\min}, I_{\max})$$

$$d_{\text{raw}} = -(y_m[k] - y_m[k-1]) / T_s$$

$$D[k] = (\tau_d \cdot D[k-1] + T_s \cdot d_{\text{raw}}) / (\tau_d + T_s)$$

$$u[k] = \text{clamp}(K_p \cdot e[k] + I[k] + K_d \cdot D[k], u_{\min}, u_{\max})$$

Key design decision: the derivative is computed on the *measurement* $y_m[k]$, not the error $e[k]$. This eliminates the **derivative kick**: when the setpoint $r[k]$ steps, the error jumps instantly but the measurement changes smoothly — so the derivative does not

spike.

Sample rate selection

Shannon's theorem requires $f_s > 2 \cdot f_{\max}$. In control this is not sufficient: computational delay and anti-aliasing demand more margin. The engineering rule of thumb:

$$f_s \geq 10 \times f_{\text{closed-loop bandwidth}}$$

Application	Typical BW	Minimum f_s	Typical T_s
Temperature regulation	0.1-1 Hz	10 Hz	100 ms
Gait / IMU (medical wearable)	5-20 Hz	200 Hz	5 ms
DC motor velocity control	20-100 Hz	1 kHz	1 ms
Power converter (current)	1-5 kHz	50 kHz	20 μ s

§ 06 – STABILITY

Stability Analysis

A control system is stable if and only if bounded inputs produce bounded outputs (BIBO stability). In the z -plane, stability requires all closed-loop poles to lie strictly **inside the unit circle** $|z| < 1$.

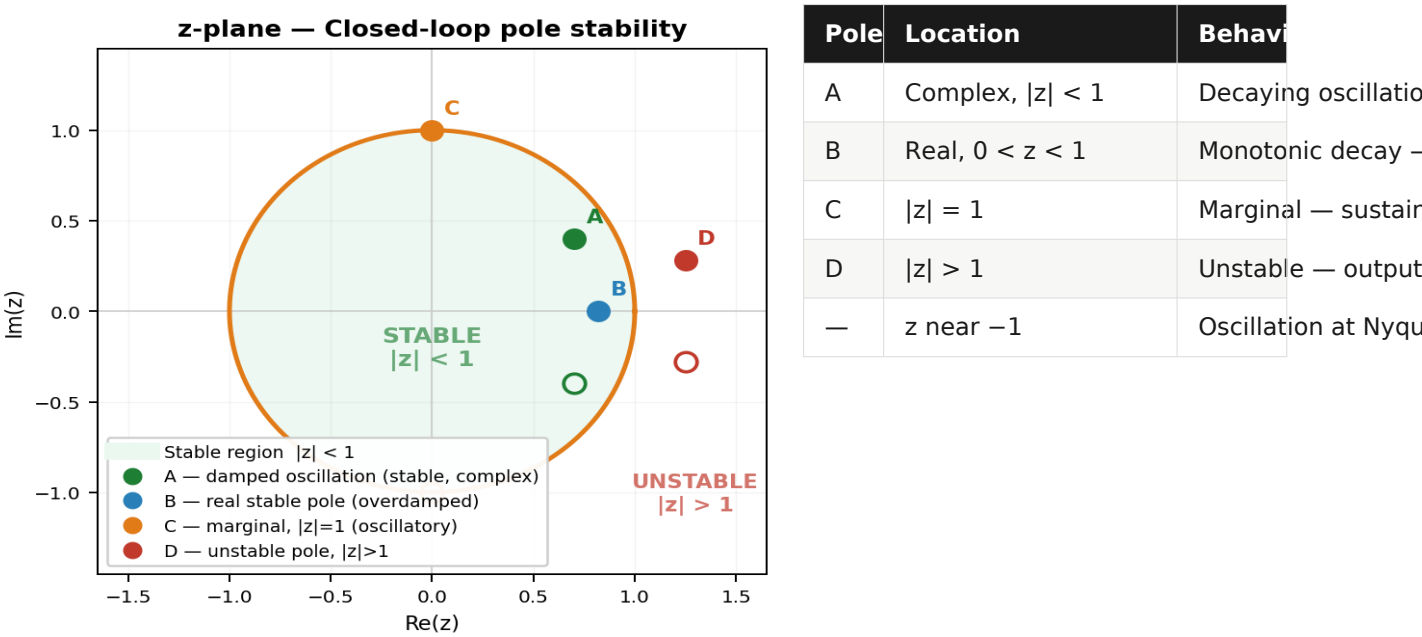


Figure 2 — z -plane stability map. Green shaded region = stable. Poles A-D illustrate different dynamic behaviours.

Gain margin and phase margin (Bode)

For continuous-time design, the open-loop frequency response $L(j\omega) = C(j\omega) \cdot G(j\omega)$ reveals how close the closed loop is to instability. Two key robustness metrics:

Margin	Definition	Minimum safe value
Gain Margin (GM)	Additional gain factor before instability; measured at phase crossover ω_{pc} where $\angle L = -180^\circ$	phase margin of 20°
Phase Margin (PM)	Additional phase lag before instability; measured at gain crossover ω_{gc} where $ L = 1$	gain margin of 6 dB (prefer 10 dB)

$$GM = -20 \cdot \log_{10} |L(j\omega_{pc})| \text{ [dB]} \cdots PM = 180^\circ + \angle L(j\omega_{gc})$$

Firmware stability check

Runtime heuristic: if $|u[k]|$ saturates to `out_max` for more than N consecutive samples AND the error $e[k]$ is not decreasing in magnitude, something is wrong — excessive gain, plant model mismatch, or sensor failure. Implement a saturation counter and safe-state fallback (zero output, raise fault flag, alert watchdog).

§ 07 – FIRMWARE ENGINEERING

Embedded Implementation Concerns

01 Anti-Windup

When the actuator saturates (PWM at 100%, valve fully open), the integrator accumulates without bound — *windup*. On release the system over-corrects violently. Three complementary fixes: (1) **Clamping**: limit $I[k]$ directly to $[I_{\min}, I_{\max}]$. (2) **Back-calculation**: subtract a proportion of the saturation error back into the integrator. (3) **Conditional integration**: freeze $I[k]$ when the output is saturated AND the error would push it further. The production code uses all three.

02 Derivative Kick Prevention

A sudden setpoint step creates a spike in de/dt that drives the derivative term to a huge instantaneous value. Solution: compute the derivative on the *measurement* $-dy_m/dt$ instead of the error de/dt . A setpoint step now has no effect on the derivative term — only the smooth plant dynamics appear in it.

03 Derivative Noise Filtering

Raw differentiation has transfer function $K_d \cdot s$, which amplifies noise by $K_d \cdot \omega$. Always pair with a first-order LPF: $K_d \cdot s / (\tau_d \cdot s + 1)$. Choose τ_d so the corner frequency $f_c = 1 / (2\pi \cdot \tau_d)$ is above the control bandwidth but below the sensor noise floor. Rule of thumb: $\tau_d = K_d / (5 \text{ to } 10)$.

04 Bumpless Transfer (Manual → Auto)

When switching from manual to automatic mode, pre-load the integrator so the first auto-mode output equals the current manual output: $I_0 = u_{\text{manual}} - K_p \cdot e[0]$. No bump, no abrupt actuator step. Critical in medical devices where a sudden step output could cause patient harm or mechanical damage.

05 Floating-Point vs. Fixed-Point

Cortex-M4/M7 and Xtensa LX7 (ESP32-S3) have a hardware FPU — use *float* freely. Cortex-M0/M0+ has no FPU — every float operation is a software trap (50–100 cycles). Use Q15 or Q31 fixed-point arithmetic instead. Q15 multiply: $\text{result} = (\text{int32_t})(a * b) \gg 15$. IEC 62304 requires documenting the numeric format and overflow behaviour as a software design record.

06 Timing Determinism (WCET)

The control loop must execute in bounded time every T_s . Use a hardware timer feeding a semaphore to the control thread — *not* `k_sleep()` or `vTaskDelay()` which have RTOS

scheduler jitter. Profile with a logic analyser or GPIO toggle. Worst-case execution time (WCET) must be well under T_s . IEC 62304 Class B/C requires WCET analysis for safety-critical loops.

07 Sample Rate Jitter

If T_s varies between samples, the discrete derivative $(y[k]-y[k-1]) / T_s$ is corrupted and the integral accumulates at the wrong rate. Measure the actual interval with a hardware timer counter and use it in the difference equations, or — simpler — guarantee deterministic T_s via a dedicated hardware timer ISR + high-priority RTOS thread.

08 Sensor Validation

Before feeding any measurement into the PID, validate: (1) Range check — is the value physically possible? (2) Rate-of-change check — did it jump 10× in one sample (dropout/glitch)? (3) Health flag — is the sensor reporting a fault code via I²C/SPI? Feeding a corrupted value into a running integrator drives the actuator to saturation within seconds.

§ 08 – RTOS

RTOS Integration Pattern

In Zephyr RTOS or FreeRTOS, the canonical pattern for a real-time control loop is: **hardware timer** → **semaphore** → **control thread**. This decouples the timing interrupt (ISR context) from the computation (thread context), allowing the RTOS scheduler to handle priorities correctly and giving deterministic bounded execution.

Pattern walkthrough

Step	Actor	What happens
1	Hardware timer	Fires every exactly Ts. Configured in a dedicated timer peripheral (k_timer in Zephyr, xTimer in FreeRTOS).
2	Timer ISR	Timer expiry callback executes in ISR context. Does exactly one thing: calls k_sem_give() (or xSemaphoreGive() in FreeRTOS).
3	RTOS scheduler	Semaphore give wakes the control thread if it was waiting. Scheduler preempts any lower priority threads.
4	Control thread	Takes the semaphore (k_sem_take). Reads sensor. Calls pid_update(). Writes actuator. Releases semaphore.

Thread priority assignment (Zephyr)

Thread	Priority	Rationale
Control loop	2 (high preemptible)	Highest application priority. Must execute within Ts.
BLE GATT handler	5	Real-time but tolerates occasional missed packet.
Sensor post-processing	7	Batch buffering; not timing critical for control.
Cloud telemetry	10	Background; may be deferred arbitrarily.
Logging / shell	12	Lowest priority; may be starved under heavy load.

Critical ISR rule

Never run the PID directly inside the timer ISR. ISRs must be minimal. If the PID calls LOG_INF(), acquires a mutex, or performs floating-point on a no-FPU core in ISR context, you will get kernel panics, hardware faults, or priority inversions. Keep ISRs under 20 instructions. All computation belongs in the control thread.

§ 09 – SOURCE CODE

Full Annotated C Implementation

The complete production-grade implementation below covers all the theory in this tutorial. Every comment cross-references the section that justifies the design decision. The code is portable across Zephyr RTOS and FreeRTOS on any STM32, ESP32-S3, or Cortex-M / Xtensa target with a hardware FPU.

pid_controller.h — API and types

pid_controller.h

```
#ifndef PID_CONTROLLER_H
#define PID_CONTROLLER_H
#include <stdint.h>

/* §3: Configuration – fill once, pass to pid_init() */
typedef struct {
    float kp, ki, kd;          /* §3: the three PID gains */
    float ts;                  /* §5: sample period [s] */
    float tau;                 /* §3: derivative LPF time constant tau_d */
    float out_min, out_max;    /* §7: actuator output clamp */
    float int_min, int_max;    /* §7: integrator anti-windup clamp */
} pid_cfg_t;

/* Controller state – one instance per loop */
typedef struct {
    pid_cfg_t cfg;
    float integrator;          /* I[k] – accumulated integral §5 */
    float prev_meas;          /* y_m[k-1] – for derivative-on-measurement §7 */
    float d_filt;             /* D[k-1] – low-pass filtered derivative §5 */
    float out;                /* u[k] – last computed output */
    uint32_t ticks;           /* sample counter (diagnostics) */
} pid_t;

void pid_init(pid_t *pid, const pid_cfg_t *cfg);
void pid_reset(pid_t *pid); /* call before enabling loop */
float pid_update(pid_t *pid, float sp, float meas);
void pid_set_gains(pid_t *pid, float kp, float ki, float kd);
void pid_set_integrator(pid_t *pid, float val); /* bumpless transfer §7 */
#endif
```

pid_controller.c — Part 1: init and reset

pid_controller.c (1/2)

```
#include "pid_controller.h"
#include <string.h>

/* Portable clamp – no stdlib dependency (MISRA §7) */
static inline float clampf(float v, float lo, float hi) {
    if (v < lo) return lo;
    if (v > hi) return hi;
    return v;
}

void pid_init(pid_t *pid, const pid_cfg_t *cfg) {
    memcpy(&pid->cfg, cfg, sizeof(pid_cfg_t));
    pid_reset(pid);
}

void pid_reset(pid_t *pid) {
    pid->integrator = 0.0f;
    pid->prev_meas = 0.0f;
    pid->d_filt = 0.0f;
    pid->out = 0.0f;
    pid->ticks = 0u;
}

void pid_set_gains(pid_t *pid, float kp, float ki, float kd) {
    pid->cfg.kp = kp;
    pid->cfg.ki = ki;
    pid->cfg.kd = kd;
}

void pid_set_integrator(pid_t *pid, float val) {
    pid->integrator = clampf(val, pid->cfg.int_min, pid->cfg.int_max);
}
```

pid_controller.c — Part 2: pid_update core algorithm

pid_controller.c (2/2)

```

float pid_update(pid_t *pid, float sp, float meas) {
    const pid_cfg_t *c = &pid->cfg;

    /* §1: error signal */
    float err = sp - meas;

    /* §3: proportional term */
    float p_term = c->kp * err;

    /* §5/§7: integral + conditional anti-windup
     * Candidate update; freeze integrator if output is saturated
     * AND error is pushing further into saturation. */
    float int_cand = pid->integrator + c->ki * c->ts * err;
    int_cand = clampf(int_cand, c->int_min, c->int_max);
    float pre_sat = p_term + int_cand;
    if (!((pre_sat > c->out_max && err > 0.0f) ||
          (pre_sat < c->out_min && err < 0.0f))) {
        pid->integrator = int_cand;
    }

    /* §7: derivative on measurement – avoids setpoint kick
     * d_raw = -dy_m/dt (positive when output is falling toward SP) */
    float d_raw = -(meas - pid->prev_meas) / c->ts;

    /* §3/§5: Tustin first-order LPF  $H(s) = 1/(\tau_d * s + 1)$  */
    if (c->tau > 0.0f)
        pid->d_filt = (c->tau * pid->d_filt + c->ts * d_raw)
            / (c->tau + c->ts);
    else
        pid->d_filt = d_raw;

    /* Assemble u[k] and clamp to actuator limits §5 */
    float u = p_term + pid->integrator + c->kd * pid->d_filt;
    pid->out = clampf(u, c->out_min, c->out_max);
    pid->prev_meas = meas;
    pid->ticks++;
    return pid->out;
}

```

Zephyr RTOS control task — Part 1: includes, ISR, stubs

control_task.c (1/2)

```
/* §8: hardware timer -> semaphore -> control thread */
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>
#include "pid_controller.h"

LOG_MODULE_REGISTER(ctrl, LOG_LEVEL_INF);

#define CTRL_TS_MS 10u    /* 100 Hz sample rate          */
#define CTRL_PRI0  2      /* high preemptible thread */
#define CTRL_STACK 1024u

static K_SEM_DEFINE(ctrl_sem, 0, 1);
static K_THREAD_STACK_DEFINE(ctrl_stack, CTRL_STACK);
static struct k_thread ctrl_td;
static struct k_timer ctrl_timer;

static volatile float g_setpoint = 0.0f;
static volatile bool  g_enabled  = false;

/* §8: ISR – minimal, only signals the thread.
 * Never run PID or LOG_INF in ISR context. */
static void timer_expiry(struct k_timer *t) {
    ARG_UNUSED(t);
    k_sem_give(&ctrl_sem);
}

/* Driver stubs – replace with real ADC / PWM calls */
static float read_sensor(void) { return 0.0f; }
static void write_actuator(float u) { (void)u; }
```

Zephyr RTOS control task — Part 2: thread, init, public API

control_task.c (2/2)

```

static void ctrl_thread(void *p1, void *p2, void *p3) {
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);
    pid_t pid;
    pid_init(&pid, &(pid_cfg_t){
        .kp = 2.0f, .ki = 0.5f, .kd = 0.3f,
        .ts = CTRL_TS_MS / 1000.0f,
        .tau = 0.05f, /* §3: derivative LPF */
        .out_min = -100.0f, .out_max = 100.0f,
        .int_min = -50.0f, .int_max = 50.0f,
    });

    while (true) {
        k_sem_take(&ctrl_sem, K_FOREVER); /* block until Ts */

        if (!g_enabled) {
            write_actuator(0.0f);
            pid_reset(&pid); /* §7: safe state */
            continue;
        }
        float meas = read_sensor();
        float u = pid_update(&pid, g_setpoint, meas);
        write_actuator(u);

        if ((pid.ticks % 100u) == 0u)
            LOG_DBG("sp=%.3f meas=%.3f u=%.3f",
                    (double)g_setpoint, (double)meas, (double)u);
    }
}

int ctrl_init(void) {
    k_timer_init(&ctrl_timer, timer_expiry, NULL);
    k_timer_start(&ctrl_timer,
                  K_MSEC(CTRL_TS_MS), K_MSEC(CTRL_TS_MS));
    k_thread_create(&ctrl_td, ctrl_stack,
                   K_THREAD_STACK_SIZEOF(ctrl_stack),
                   ctrl_thread, NULL, NULL, NULL,
                   CTRL_PRIO, 0, K_NO_WAIT);
    k_thread_name_set(&ctrl_td, "ctrl");
    return 0;
}

void ctrl_set_setpoint(float sp) { g_setpoint = sp; }
void ctrl_enable(bool en) { g_enabled = en; }
float ctrl_get_output(void) { return 0.0f; }

```

Next steps

This PID module is the foundation. Natural extensions: (1) Cascade PID — inner current loop + outer velocity loop for motor drives. (2) State-space / LQR — optimal state feedback via Riccati equation; a direct extension of your ESKF sensor fusion work. (3) Kalman / EKF observer — estimated states for unmeasured plant variables. (4) MPC — Model Predictive Control for multi-variable plants with hard constraints.